Spring JDBC Template

Course: Full Stack Development with Java / Advanced Java
Module: Spring JDBC Template
Duration: 6–8 Lectures + Laboratory Sessions

Table of Contents

Lecture 1: Introduction to Spring Framework

What is Spring?

Spring is one of the most popular Java frameworks used for developing enterprise-level applications.

It provides:

- Dependency Injection (DI)

- Inversion of Control (IoC)
- JDBC Support
- Transaction Management
- Spring MVC
- Spring Boot
- Security
- REST API Development

Spring makes Java application development easier by reducing boilerplate code.

What is JDBC?

JDBC (Java Database Connectivity) is a Java API that allows Java programs to communicate with databases.

Using JDBC, we can:

- Connect to database
- Execute SQL queries
- Insert data
- Update data
- Delete data
- Retrieve data

Traditional JDBC Example

```
Connection con = DriverManager.getConnection(url,user,password);

PreparedStatement ps =
con.prepareStatement("select * from employee");

ResultSet rs = ps.executeQuery();

while(rs.next()){

    System.out.println(rs.getString(2));

}

rs.close();
ps.close();
con.close();
```

Problems with Traditional JDBC

Ask students:

How many lines of code are required to execute one SQL query?

Answer:

- Load Driver
- Create Connection
- Create PreparedStatement
- Execute Query
- Process ResultSet
- Close ResultSet
- Close Statement
- Close Connection

Almost **15–20 lines of repetitive code**.

Problems:

- Boilerplate Code
- Resource Management
- Exception Handling
- Difficult Maintenance
- More Chances of Memory Leak

Why Spring JDBC?

Spring JDBC removes unnecessary JDBC code.

Instead of writing

```
Connection

PreparedStatement

ResultSet

finally

close()
```

Spring writes all these internally.

Developer only writes SQL.

Traditional JDBC vs Spring JDBC

Traditional JDBC	Spring JDBC
More Code	Less Code
Manual Connection Handling	Automatic
Manual Exception Handling	Automatic
Manual Resource Closing	Automatic
Difficult to Maintain	Easy
Time Consuming	Fast Development

What is JdbcTemplate?

JdbcTemplate is the heart of Spring JDBC.

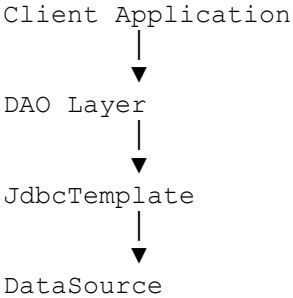
It is a helper class that performs all database operations.

It internally performs:

- Open Connection
- Create Statement
- Execute SQL
- Handle Exceptions
- Close Resources

Developer only provides SQL.

Spring JDBC Architecture



Database

Components of Spring JDBC

1. DataSource

Responsible for creating database connections.

Example

```
DriverManagerDataSource ds =
new DriverManagerDataSource();
```

---

2. JdbcTemplate

Executes SQL Queries.

Example

```
JdbcTemplate jdbcTemplate =
new JdbcTemplate(dataSource);
```

---

3. RowMapper

Converts one database row into one Java Object.

Database

```
101 Rohan 50000 Manager
```

↓

Java Object

```
Employee
```

---

4. DAO Layer

DAO = Data Access Object

Responsible for database operations.

Example

```
save()

update()

delete()

findAll()

findById()
```

Spring JDBC Workflow

```
Application

↓

DAO

↓

JdbcTemplate

↓

DataSource

↓

Database
```

Project Structure

```
SpringJDBC

│

├── config

│       SpringConfig.java

│

├── entity

│       Employee.java

│
```

```
├── mapper
│       EmployeeRowMapper.java
│
├── dao
│       EmployeeDAO.java
│       EmployeeDAOImpl.java
│
└── AppMain.java
```

DataSource Configuration

```
@Bean
public DataSource dataSource(){

DriverManagerDataSource ds=
new DriverManagerDataSource();

ds.setDriverClassName(
"oracle.jdbc.OracleDriver");

ds.setUrl(
"jdbc:oracle:thin:@localhost:1521:XE");

ds.setUsername("system");

ds.setPassword("oracle");

return ds;

}
```

JdbcTemplate Bean

```
@Bean
public JdbcTemplate jdbcTemplate(DataSource ds){

return new JdbcTemplate(ds);

}
```

Employee Entity

```
public class Employee{

private int empId;

private String empName;

private double salary;

private String jobTitle;

}
```

CRUD Operations

Insert

```
jdbcTemplate.update(

"insert into employee values(?,?,?,?)",

101,

"Rohan",

50000,

"Manager");
```

Update

```
jdbcTemplate.update(

"update employee set salary=? where emp_id=?",

70000,

101);
```

Delete

```
jdbcTemplate.update(

"delete from employee where emp_id=?",

101);
```

Retrieve One Record

```
Employee emp=

jdbcTemplate.queryForObject(

"select * from employee where emp_id=?",

new EmployeeRowMapper(),

101);
```

Retrieve Multiple Records

```
List<Employee> list=

jdbcTemplate.query(

"select * from employee",

new EmployeeRowMapper());
```

Understanding RowMapper

What is RowMapper?

RowMapper converts each row of ResultSet into a Java object.

Database

```
101

Rohan

50000
```

↓

Employee Object

```
Employee

id=101

name=Rohan
```

RowMapper Code

```
public class EmployeeRowMapper
implements RowMapper<Employee>{

public Employee mapRow(
ResultSet rs,
int rowNum)
throws SQLException{

Employee emp=
new Employee();

emp.setEmpId(rs.getInt("EMP_ID"));

emp.setEmpName(rs.getString("EMP_NAME"));

emp.setSalary(rs.getDouble("SALARY"));

return emp;

}

}
```

BeanPropertyRowMapper

No custom RowMapper required.

```
jdbcTemplate.query(

"select * from employee",

new BeanPropertyRowMapper<>(
Employee.class));
```

Automatically maps

Database Column

```
EMP_NAME
```

↓

Java Variable

```
empName
```

ResultSetExtractor

Processes the entire ResultSet at once.

Useful when

- Parent Child Objects
- Complex Queries
- Nested Objects

NamedParameterJdbcTemplate

Instead of

```
insert into employee values(?,?,?)
```

Use

```
insert into employee
values(:id,:name,:salary)
```

Advantages

- Easy to Read
- Better Maintenance
- Less Mistakes

Batch Update

Insert multiple records together.

```
jdbcTemplate.batchUpdate(...)
```

Advantages

- Faster
- Less Network Calls
- Better Performance

Transactions

Suppose money transfer

```
Account A

↓

Withdraw

↓

Account B

↓

Deposit
```

If Deposit fails

↓

Rollback Withdraw

This is Transaction Management.

Exception Handling

Spring converts SQL exceptions into

```
DataAccessException
```

Common Exceptions

- DuplicateKeyException
- EmptyResultDataAccessException
- BadSqlGrammarException

Advantages of Spring JDBC

- Less Coding
- Automatic Resource Management
- Automatic Exception Handling

- Faster Development
- Better Readability
- Easy Testing
- Supports Transactions
- High Performance

When to Use Spring JDBC?

Use Spring JDBC when:

- SQL is complex
- High Performance is required
- Full control over SQL
- Stored Procedures are used
- Reporting applications

Spring JDBC vs Hibernate

Spring JDBC	Hibernate
SQL Required	No SQL Required
Faster	Slightly Slower
Manual Mapping	Automatic Mapping
Better for Reports	Better for Large Applications
Lightweight	ORM Framework

Mini Project

Employee Management System

Modules

- Employee CRUD
- Department CRUD
- Salary Management
- Search Employee
- Batch Insert
- Transaction Management

Laboratory Exercises

1. Configure Spring JDBC.
2. Connect Oracle Database.
3. Create Employee Table.
4. Insert Records.
5. Update Employee Details.
6. Delete Records.
7. Display All Employees.
8. Search Employee by ID.
9. Implement `RowMapper`.
10. Implement `BeanPropertyRowMapper`.
11. Use `NamedParameterJdbcTemplate`.
12. Batch Insert Employees.
13. Department CRUD.
14. Employee–Department Join Query.
15. Transaction Example.

Interview Questions

1. What is Spring JDBC?
2. Why do we use `JdbcTemplate`?
3. What is the role of `DataSource`?
4. What is a `RowMapper`?
5. Difference between `query()` and `queryForObject()`?
6. What is `BeanPropertyRowMapper`?
7. What is `NamedParameterJdbcTemplate`?
8. Explain batch updates in Spring JDBC.
9. How does Spring handle SQL exceptions?
10. Compare Spring JDBC and Hibernate.

Summary

At the end of this module, students should understand:

- The limitations of traditional JDBC.
- How Spring JDBC simplifies database programming.
- The role of `JdbcTemplate`, `DataSource`, and `RowMapper`.
- How to perform CRUD operations efficiently.

- When to choose Spring JDBC over Hibernate.
- Best practices for building maintainable database applications.